

WEBCAT: Web-based Code Assurance and Transparency

Giulio Berra

Vrije Universiteit Amsterdam

Amsterdam, NL

giulio@lsd.cat

ABSTRACT

Ensuring code integrity in browser-based applications remains a longstanding challenge exacerbated by the complexity of modern web environments. We propose Web-based Code Assurance and Transparency, a novel code integrity verification and enforcement mechanism that prevents the execution of unverified code, unlike previous approaches premised on user-visible error indicators or permissive failure modes. WEBCAT remains compatible with modern web features, uses existing cryptographic components without reinventing primitives or requiring expensive infrastructure, and provides verifiable logs of all system components, even under degraded operational conditions. It follows a separation-of-concerns model in which hosting providers require no special trust or cryptographic keys to deploy developer-signed applications, reflecting real-world deployment scenarios in which trusted applications may be served by multiple less-trusted hosts.

We evaluate our approach by porting Jitsi, GlobaLeaks, Element, CryptPad, Standard Notes, and Bitwarden, demonstrating compatibility across a diverse set of applications. Benchmark results indicate an overhead of up to 2% for non-enrolled domains on cold starts and up to 20% for enrolled ones. Under warm start conditions, the overhead reaches 25% for enrolled domains and 5% for non-enrolled ones—lower than previous methods while addressing a larger threat model and remaining compatible with existing applications.

1 INTRODUCTION

The issue of browser code integrity has long been an open problem: in contrast to prepackaged applications, an end user relying on browser-based applications must trust that the code that is served to them is authentic every single time they interact with a given website or service. Privacy-preserving platforms and privacy advocacy organizations, such as Riseup, Autistici or Disroot, strive to provide usable encryption [21], such as in emails, collaboration and organization tools, as well as federated services such as Mastodon or Matrix. But no matter the type of encryption these platforms provide, when implemented by a web-based client, they present a large attack surface: legally mandated backdoors¹, server or domain takeover or seizures, and even BGP hijackings. An integrity-checking mechanism for applications that perform browser-based encryption would benefit both end users and website administrators.

Encrypted messaging applications in particular have acknowledged this problem, taking different approaches to address it. Meta’s Code Verify project, explored in Section 3.5, is meant to address some of the risks of using web-based versions of Meta’s messaging

platforms, such as WhatsApp Web, while Signal² has no plans to offer a web-based version of its application³.

In contrast to broadly-applicable web application security mechanisms such as the Content Security Policy (CSP), this project provides a complete integrity-checking mechanism for a security-critical subset of web applications. These applications differ from traditional web services, because their core security property is to protect the confidentiality of users’ information from even the web service providers hosting them.

In a typical web application, the website backend is the ultimate receiver and maintainer of user information, which is rendered accessible and editable via the application; this is particularly evident when the website serves predominantly as a frontend for end users, such as an online banking platform, a client portal at a health institution, a government services website, or a newspaper. In the case of encrypted backup tools⁴, collaborative office suites and online pads⁵, or web-based encrypted communications platforms⁶, however, the intended receivers of the information protected by browser-based encryption are either the users themselves or are other peers, and not the web service provider; this distinction informs our project’s design goals.

WEBCAT has four main components: a signing script, allowing application developers to generate signed application manifests; an enrollment server where site owners enroll participating domains; an updater service that builds a list of trusted signers per domain; and an in-browser integrity checking mechanism. The system is privacy-preserving and end-user friendly, and serves to decouple trust from a web server from trust in the client-side code that it is serving. This represents a meaningful contribution to a decentralized and encrypted web, the relevance and necessity of which, in the author’s view, is underscored by the current political climate.

WEBCAT provides an auditable and transparent code signing mechanism for web applications without increasing their need for client-side persistence or placing additional demands on end users. It is compatible both with open- and closed-source web applications⁷; when the integrity chain from developers to end users is preserved, end users can make an independent determination about which services to trust. While code integrity enforcement for client web applications is the primary design goal, WEBCAT could also

¹For example, as it was requested to encrypted email provider Tuta in 2020: <https://www.heise.de/news/Gericht-zwingt-Mailprovider-Tutanota-zu-Ueberwachungsfunktion-4972460.html>

²<https://signal.org>

³<https://github.com/signalapp/Signal-Desktop/issues/4466>

⁴For example, <https://mega.io/>

⁵For example, <https://cryptpad.fr/>

⁶For example, <https://wire.com/>

⁷While the concept of a closed-source browser-based application is relative, since all code must be available to the browser for it to run, there are still differences in transparency and auditability; if the source code is not offered in an easily-readable format, code quality and security assessments, such as whether a vulnerability has been fixed or a new one introduced, become difficult and time-consuming.

support other use cases in which end users could benefit from additional integrity assurances, such as on signed contact pages or canary statements⁸.

2 BACKGROUND

Various existing security and authentication measures have been implemented to offer improvements to web application security and to address different points at which content failing particular integrity checks could be delivered to an end user; examples include Transport Layer Security, Sub-Resource Integrity, and the Content Security Policy, though all are distinct in terms of purposes and architecture. However, in the scenario of a compromised server serving malicious code to a subset of its user base, none of these mechanisms would sufficiently address the issue of browser-based code integrity.

A further hindrance to the development of integrity-checking mechanisms for browser-based applications is that the conceptual isolation between code and data in web application has always been thin [11], making isolation and verification of browser code more difficult. Certain paradigms, such as the now-deprecated use of JSONP response scripts, have even encouraged mixing code and data; prior to the introduction of Cross-Origin Resource Sharing headers, JSONP was previously recommended to allow access to cross-origin data, bypassing the browser Same-Origin Policy [7]. Some applications mix the two in other ways, for example, self-hostable applications that support per-instance personalization via the direct editing of HTML files, or applications that write configuration as JavaScript files that are then sourced either statically or via dynamic endpoints, discussed further in Section 7.

All of these practices conflict with the idea of data isolation: generally, configuration information should be in a structured data format parsed at runtime, not sourced as code. Approaches that can address the need for per-instance configuration while ensuring that executable content can be subjected to verification checks include, for example, using static assets such as images to achieve visual customizations and branding; using a JavaScript-based templating language; or by dynamically writing HTML in the DOM⁹.

2.1 The Update Framework

The Update Framework (TUF) [25] is a security architecture that provides an abstracted, flexible framework for securing software updates, in particular to resist a range of attacks such as key compromise or repository tampering. At the core of TUF is update metadata, which are verifiable records including not just the target files for a given update, but also authorized signing parties and freshness timestamps. TUF introduces a role-based trust model that delegates responsibilities across multiple cryptographic keys, enabling key rotation and limiting the consequences of individual key exposure. It employs mechanisms such as versioning, expiration, and threshold signatures in order to recover from key compromises and detect freezing attacks. Algorithm 1 presents a simplified, high-level version of the procedure required to update a target file, identified by the variable *path*. The function *UpdateRoot* is a core

component of TUF that updates the client's trusted root keys by sequentially verifying newer root metadata, even when starting from outdated or expired versions.

Algorithm 1 TUF.UpdateTarget

Require: Target path *path*, Trusted root *root*, Current time *ts*

```

1: procedure UPDATETARGET(path, root, ts)
2:   root ← UPDATEROOT(root, ts)
3:   timestamp ← FETCH('timestamp.json')
4:   if ¬ VERIFY(timestamp, root) then
5:     ERR('invalid signature on timestamp metadata')
6:   end if
7:   if timestamp.expiration > ts then
8:     ERR('expired timestamp metadata')
9:   end if
10:  snapshot ← FETCH('snapshot.json')
11:  if ¬ VERIFY(snapshot, root) then
12:    ERR('invalid signature on snapshot metadata')
13:  end if
14:  if snapshot.expiration > ts then
15:    ERR('expired snapshot metadata')
16:  end if
17:  targets ← FETCH('targets.json')
18:  if ¬ VERIFY(targets, root) then
19:    ERR('invalid signature on targets metadata')
20:  end if
21:  if targets.expiration > ts then
22:    ERR('expired targets metadata')
23:  end if
24:  if ISDELEGATED(path, targets) then
25:    delegation ← FETCH(path)
26:    if ¬ VERIFY(delegation, root) then
27:      ERR('invalid signature on delegation metadata')
28:    end if
29:    if delegation.expiration > ts then
30:      ERR('expired delegation metadata')
31:    end if
32:    desc ← FINDTARGET(path, delegation)
33:  else
34:    desc ← FINDTARGET(path, targets)
35:  end if
36:  if desc is NULL then
37:    ERR('target not found in metadata')
38:  end if
39:  file ← FETCH(path)
40:  if H(file) ≠ desc.hash then
41:    ERR('target file hash mismatch')
42:  end if
43:  return file
44: end procedure

```

⁸For example, <https://www.qubes-os.org/security/canary/>

⁹While HTML is non-executable, we consider it for the sake of this project; see Section 9 for some considerations around the tradeoff of this choice.

2.2 Sigstore

Sigstore [23] is one of the most widely adopted systems for signing code, packages, and artifacts, employed for supply chain security in a multitude of environments, including npm¹⁰, Docker, and Python¹¹. Sigstore’s goal is to allow developers to sign artifacts without having to manage long-lived keys and certificates, avoiding the associated complexities. While every component of Sigstore is open source, and client libraries are actively maintained in languages including Go, TypeScript, and Python,¹² running the entire dedicated infrastructure can be expensive, as it requires highly available services such as transparency logs and at least one certificate authority. The *Public Good* instance, as it is named, is maintained as a cooperative effort by multiple stakeholder companies, such as Red Hat and Google¹³. A schematic overview of the signing process from a developer perspective is charted in Figure 1.

Sigstore is composed of two main services: Fulcio, a certificate authority, and Rekor, a transparency log that supports different types of objects¹⁴. When a developer wants to sign an artifact, the first step is to request a Fulcio-issued certificate for the identity being claimed. Fulcio verifies the developer’s identity by requiring a one-time OIDC login via a supported authorization provider. If self-hosting, it is theoretically possible to support any OIDC provider, though the *Public Good* instance currently supports only GitHub, Google, and Microsoft. If the OIDC login is successful (or a previously obtained identity token is provided and valid), Fulcio signs a pre-certificate, which is not yet usable by clients, and sends it to Rekor for inclusion. Rekor returns to Fulcio a Signed Certificate Timestamp (SCT), which is a signed promise of inclusion in the log. Fulcio appends the SCT to the certificate, signs the definitive version, and logs it in Rekor¹⁵.

The certificates issued by Fulcio are short-lived, usually lasting only a few minutes, and are considered ephemeral since private keys can be discarded after the signing event. Developers should request new certificates every time they perform a signing event, whether for a single artifact or multiple artifacts, as long as all operations fit within the certificate’s validity window. Thus, developers do not need to store, encrypt, or protect keys in any specific way beyond keeping their signing machines and online identities secure. By default, the certificate keys are kept in memory and never written to disk to avoid the risk of backups or stale temporary files.

Once a developer has obtained a certificate, they can produce a signature over an artifact, such as a release tarball or executable. The signing process involves cryptographically signing a hash of the artifact’s content using the private key corresponding to the issued certificate. After signing, the artifact hash, the signature, and the public key are sent to Rekor to log the signing event. In case of a successful submission, Rekor returns an inclusion proof, which includes the tree size, the new root hash, a signature over it using a Rekor-specific key (constituting the signed tree head, STH), and the Merkle tree leaf hashes necessary to reconstruct the root

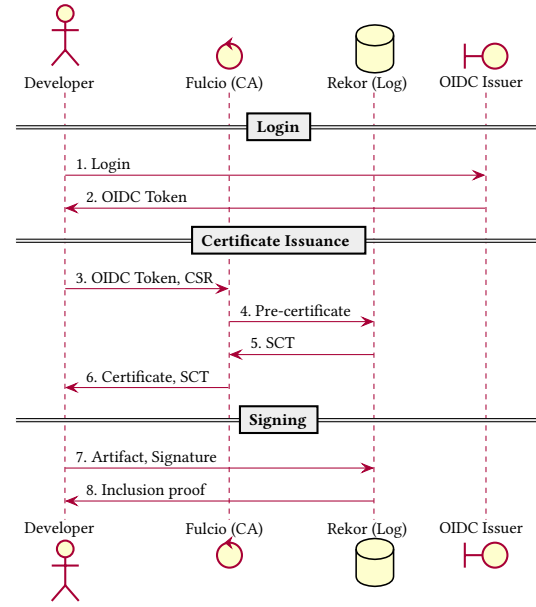


Figure 1: Sigstore signing procedure.

hash using the newly included leaf hash. Lastly, when the developer publishes the release, they must include their signing certificate, the signature, and the Rekor inclusion proof. Clients can perform offline verification using Algorithm 2.

A Sigstore bundle thus contains all the necessary components to fully verify the trust chain offline: the signing certificate, including the SCT, the signature, and the inclusion proof or promise, or both. Furthermore, there are tools available for monitoring Rekor logs: developers can receive alerts when a certificate is issued for their monitored identities or when one of their past or current certificates is used in a signing event, allowing them to take appropriate action in case of detected misuse. Clients also require the trust material for Fulcio and Rekor, which is distributed via TUF. To securely bootstrap this process, a current or previously trusted TUF root metadata file must include a delegation to the Sigstore trust metadata. With this delegation in place, both TUF metadata and the Sigstore trust roots can be securely updated at client startup.

2.3 Sigsum

Sigsum [22] is a lightweight and general-purpose transparency log system designed to provide cryptographic proof of key usage. Unlike transparency logs such as the Certificate Transparency Log of the TLS ecosystem [26], or Rekor for Sigstore [23], Sigsum is a general-purpose log that allows the public logging of any signature based on the Ed25519 scheme [5]. It only stores the essential information to keep the tree consistent, which include the SHA-256 hash of the content (defined as the checksum, 32 bytes), the SHA-256 hash of the signing public key (32 bytes), and the signature of the public key over the content (64 bytes). Upon submission, the Sigsum server hashes the content and the submitter’s public key independently, and verifies the signature. If that is successful, and if the submitter has completed any other checks that might be

¹⁰ <https://docs.npmjs.com/generating-provenance-statements>

¹¹ <https://www.python.org/downloads/metadata/sigstore/>

¹² https://docs.sigstore.dev/language_clients/language_client_overview/

¹³ <https://openssf.org/blog/2023/10/03/running-sigstore-as-a-managed-service-a-tour-of-sigstores-public-good-instance/>

¹⁴ <https://docs.sigstore.dev/about/tooling/>

¹⁵ https://docs.sigstore.dev/certificate_authority/certificate-issuing-overview/

Algorithm 2 Sigstore.Verify

Require: Trusted TUF root metadata *root*, Signature *sig*, Artifact *a*, Signing Certificate *cert*, Rekord Inclusion Proof *proof*, Fulcio CA *ca*

```

1: procedure SIGSTORE.VERIFY(sig, a, cert, proof)
2:   if  $\neg$ X509.VERIFYca(cert) then
3:     return ERR("invalid certificate")
4:   end if
5:   if  $\neg$ FULCIO.VERIFYSCT(cert.sct) then
6:     return ERR("invalid or missing SCT")
7:   end if
8:   (sth, leaf, inc, ts)  $\leftarrow$  proof
9:   artifactHash  $\leftarrow$  H(a)
10:  if artifactHash  $\neq$  leaf.checksum then
11:    return ERR("artifact hash mismatch")
12:  end if
13:  if H(cert.pk)  $\neq$  leaf.keyhash then
14:    return ERR("certificate key hash mismatch")
15:  end if
16:  if  $\neg$ VERIFYcert.pk(artifactHash, sig) then
17:    return ERR("invalid signature over artifact")
18:  end if
19:  if ts  $\notin$  cert.validityWindow then
20:    return ERR("invalid time for signature")
21:  end if
22:  if  $\neg$ VERIFYrekord.pk(sth.root, sth.signature) then
23:    return ERR("invalid Rekord signature over STH")
24:  end if
25:  leafHash  $\leftarrow$  H(leaf)
26:  if  $\neg$ REKOR.VERIFYINCPROOF(leafHash, inc, sth.root) then
27:    return ERR("invalid inclusion proof")
28:  end if
29:  return OK
30: end procedure

```

intended for rate-limiting or spam prevention, the leaf is merged into the tree and a new Signed Tree Head (STH) is produced. Sigsum's design has significantly focused on reducing the footprint of leaves, in order to be more scalable, and to prevent any submitter's chosen data from being included in the public log. The reason for the latter is a longstanding problem with publicly accessible transparency logs: leaves can never be removed without breaking the tree, and as such adversaries with denial of service intentions might attempt to include illegal or controversial content that could require a takedown by law¹⁶.

Sigsum is compatible with the Witness Protocol¹⁷, and has a policy-based system to verify offline inclusion proofs and witness cosignatures to protect against split-view attacks. The use of witness cosigning, as introduced in [28], facilitates the prevention of attacks rather than reactions to them. This allows to avoid using a gossip protocol, which would pose significant integration

challenges when faced with application-agnostic signing and verification such as in this case. Figure 2 summarizes the submission process. The Sigsum server is responsible for collecting the cosignatures as specified in the submitter's policy, and returns everything necessary to satisfy it in a single bundle that can then be distributed to clients for verification.

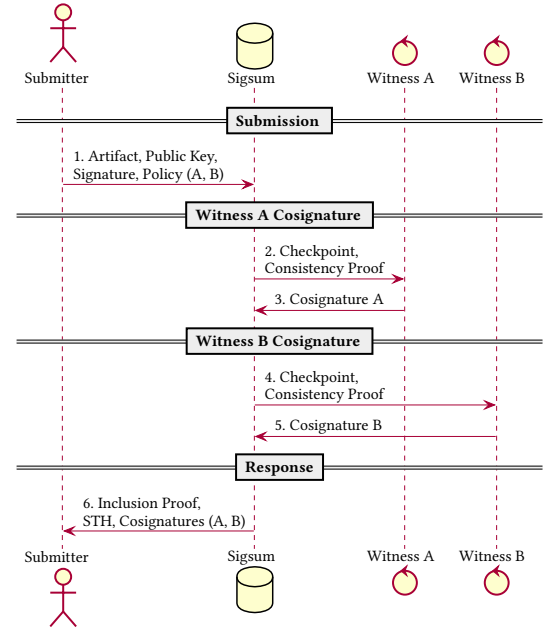


Figure 2: Sigsum submission procedure.

An example Sigsum submission process, with a policy requiring cosignatures from two witnesses (A and B), is illustrated in Figure 2. Offline proof verification is detailed in Algorithm 3.

2.4 Web Cryptography API

The Web Cryptography API is a W3C-defined standard¹⁸ that provides browser-based environments with native cryptographic primitives including hashing, key agreement, symmetric and asymmetric encryption, key derivation and secure pseudorandom number generation¹⁹. Almost all API methods are asynchronous, meaning they do not block the main JavaScript thread, although they can be waited on. Support for ECDSA and RSA asymmetric algorithms is standardized; the use of Ed25519 keys, though still in draft specification²⁰, is already supported by all major browsers²¹.

Despite offering a robust cryptographic API to web applications, the Web Cryptography API does not fundamentally change the operating model of browser-based applications [12], since the files invoking the API are still server-controlled.

¹⁶Other rights, such as the GDPR mandated "right to be forgotten," are impossible to enforce for transparency logs: <https://www.sectigo.com/resource-library/root-causes-350-public-certificates-and-the-gdpr-right-to-be-forgotten>

¹⁷<https://github.com/C2SP/C2SP/blob/tlog-checkpoint/v1.0.0-rc.1/tlog-witness.md>

¹⁸<https://w3c.github.io/webcrypto/>

¹⁹<https://developer.mozilla.org/en-US/docs/Web/API/Crypto/getRandomValues>

²⁰<https://wicg.github.io/webcrypto-secure-curves/>

²¹<https://developer.mozilla.org/en-US/docs/Web/API/SubtleCrypto/sign#ed25519>

Algorithm 3 Sigsum.Verify

Require: Artifact a , Signature sig , Submitter Public Key pk , Log Public Key $logpk$, Tree Head sth , Inclusion Proof p , Leaf $leaf$, Witness Cosignatures Σ_w , Witness Public Keys Σ_{pk} , Quorum q

```

1: procedure SIGSUM.VERIFY( $a, sig, pk, logpk, p, sth, \Sigma_w, \Sigma_{pk}, q$ )
2:    $checksum \leftarrow H(a)$ 
3:    $keyhash \leftarrow H(pk)$ 
4:   if  $leaf.checksum \neq checksum$  then
5:     return ERR("checksum mismatch")
6:   end if
7:   if  $leaf.keyhash \neq H(pk)$  then
8:     return ERR("submitter keyhash mismatch")
9:   end if
10:  if  $\neg \text{VERIFY}_{pk}(checksum, sig)$  then
11:    return ERR("invalid signature on artifact")
12:  end if
13:  if  $sth.log\_keyhash \neq H(logpk)$  then
14:    return ERR("log key hash mismatch")
15:  end if
16:  if  $\neg \text{VERIFY}_{logpk}(sth.root, sth.signature)$  then
17:    return ERR("invalid log signature on STH")
18:  end if
19:   $valid \leftarrow 0$ 
20:  for all  $(w, pk_w) \in (\Sigma_w, \Sigma_{pk})$  do
21:    if  $\text{VERIFY}_{pk_w}(sth.root, w)$  then
22:       $valid \leftarrow valid + 1$ 
23:    end if
24:  end for
25:  if  $valid < q$  then
26:    return ERR("insufficient witness cosignatures")
27:  end if
28:   $leaf\_hash \leftarrow H(leaf)$ 
29:  if  $\neg \text{VERIFYINCPROOF}(leaf\_hash, p, sth.root)$  then
30:    return ERR("invalid inclusion proof")
31:  end if
32:  return OK
33: end procedure
    
```

2.5 WebExtensions API

The WebExtensions API is a set of JavaScript-based methods that can be called from the context of a browser extension. Extensions, their assets, code, and permissions are described in a manifest file; to support cross-ecosystem portability of extensions, a generally-accepted feature set is standardized across browser vendors²². WebExtensions manifests are versioned; Manifest V2-compatible extensions allow for the blocking interception of network requests via the `webRequestBlocking` API, enabling extensions to drop or alter content before it reaches the DOM²³. This ability to intercept network requests and responses after TLS decryption and SRI checks grants its caller a privileged position from which to monitor and modify resources before they are loaded, and is commonly used by web extensions providing ad-blocking functionality.

²²<https://developer.mozilla.org/en-US/docs/Mozilla/Add-ons/WebExtensions/API>

²³<https://developer.mozilla.org/en-US/docs/Mozilla/Add-ons/WebExtensions/API/webRequest>

In recent years, Chrome has begun deprecating the V2 Manifest API in favor of a V3 Manifest API that removes the `webRequestBlocking` capability and supports only non-blocking, read-only interception²⁴. There is speculation in the wider community that this change is motivated by a desire to hinder ad blockers^{25 26 27}. Mozilla has pledged to keep V2 extension support in Firefox ESR for the foreseeable future²⁸, which consequently means that the feature will remain supported in Tor Browser as well.

2.6 Content Security Policy

The Content Security Policy (CSP) [30] is W3C-defined specification²⁹ aimed at mitigating code injection attacks by introducing a separation between executable code and data. By defining strict policies on sources from which scripts, styles, and other resources can be loaded, CSP reduces the unauthorized execution of code and narrows the attack surface of web applications. One of the most prominent features of CSP is its capacity to block dynamically loaded code and inline scripts, executing only scripts from trusted sources. CSP has expanded from script and resource constraints to encompass directives that manage frames, workers, WebSockets, and other modern web APIs to allow security policies to cover as broad an execution context as possible³⁰.

3 RELATED WORK

Various solutions to address specific aspects of integrity-checking have been proposed, though not necessarily adopted. These include a proposal for integrity-checking for file downloads [8], or an application-specific proposal for a web application that made use of client-side encryption [16]. Setting aside specificity of use-case or application, existing proposals may require application developers to significantly rewrite their applications, and generally rely on visual cues to users ("failing open") rather than on blocking content that fails particular checks.

3.1 Mylar (2014)

Mylar [24] addresses the issue of client-side code integrity, which is critical for enabling secure search over encrypted data in its client-server model. It includes a browser extension that verifies the authenticity of the main HTML page by checking a digital signature using a public key embedded in the web server's X.509 TLS certificate.

3.2 Verena (2016)

Verena [17] improves the client-server cryptographic model of Mylar, but directly relies on the same mechanism for client-side code integrity verification.

²⁴https://developer.mozilla.org/en-US/docs/Mozilla/Add-ons/WebExtensions/Chrome_incompatibilities#webrequest_api

²⁵<https://adguard.com/en/blog/firefox-manifestv3-chrome-adblocking.html>

²⁶<https://hide.me/en/blog/protect-your-privacy-after-google-disables-ad-blockers/>

²⁷<https://groups.google.com/a/chromium.org/g/chromium-extensions/c/veJy9uAwS00/m/AppqR6u-GgAJ>

²⁸<https://blog.mozilla.org/addons/2024/03/13/manifest-v3-manifest-v2-march-2024-update/>

²⁹<https://www.w3.org/TR/CSP3/>

³⁰<https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/CSP>

3.3 Page Integrity (2020)

Page Integrity³¹ is a browser extension, originally released in 2020, with the purpose of showcasing a potential solution to the code integrity problem space. The security model proposed requires web applications to be developed exclusively as a Single Page Application (SPA³²), loading all necessary logic in a single HTML page.

Developers can use public-key cryptography to sign specific HTML sources, and users are responsible for manually adding trusted public keys, or pinning specific page hashes through the extension's settings once it is installed. The authors leave open for future work on how to advance the end-user experience past a manual trust or trust-on-first-use model, as well as how to distribute out-of-band information about trusted authors or pages.

In addition, the authors mention that the only code that must be signed is the part responsible for doing the security-critical operations, such as encryption or managing secret keys. Under these circumstances, any other non-signed HTML page or unsigned executable JavaScript file would have access to all the data available under the Same Origin Policy (SOP), such as cookies, local storage and session storage.

3.4 WAIT (2021)

The approach proposed by WAIT [19] identifies four relevant parties: *Application Developers/Providers*, *Transparency Log Operators*, *Independent Auditors* and *Service Users*.

Application Developers must prepare their web application as a Single Page Application, with only one HTML file, the index page, and all JavaScript files loaded directly from that page, using Subresource-Integrity (SRI) for all of these items. Developers must also add an HTML meta tag containing their signing public key to the main HTML file. Application Developers must then submit the hash of the main HTML file and a signature over the hash using the corresponding private key to one or more transparency logging services, operated by Transparency Log Operators and most likely running as a dedicated logging service.

The transparency log(s) must queue the signature and hash for inclusion in the Merkle tree, and return a signed inclusion promise (a signed inclusion promise is a signed data structure in which the log promise to include the submitted data, similar to a PKI-related Signed Certificate Timestamp).

Application Provider(s) are responsible for the infrastructure serving the application to Service Users. Application Providers publish the signed HTML, all related assets, and the page signature, and configure a Content-Security-Policy HTTP header that does not include the `unsafe-` keyword, meaning they must avoid CSP directives such as `unsafe-inline` or `unsafe-eval`, and an additional HTTP header, `X-WAIT-Inclusion-Promise`, the value of which is the signed inclusion promise returned by the transparency log and distributed by the developer.

Service Users must have installed a browser extension that performs the final validation. This component ensures that the CSP follows the previously-specified criteria, and checks for the presence of the `X-WAIT-Inclusion-Promise` header. If the header is present, and the extension had no prior knowledge of this header

for that domain, then the domain is locally marked as enrolled in the WAIT mechanism. If the visited destination was already locally marked as part of the WAIT mechanism, and the web server no longer serves the header, then an error is thrown. If the header is present, then the inclusion proof is verified using one or more hard-coded public keys corresponding to the identities of the transparency logging services employed by the developers, and trusted by the extension. If the previous checks are successful, the page is displayed. An injected Content Script³³ is responsible for parsing the DOM and detecting whether styles or script have the integrity attribute necessary for the browser to natively enforce SRI.

3.5 Code Verify (2022)

Meta Code Verify [13] is a browser extension published by Meta for Microsoft Edge, Mozilla Firefox and Google Chrome, and is the only proposal that is distributed and maintained. Its purpose is to verify the integrity of web applications deployed by Meta for end-to-end encrypted instant messaging, namely WhatsApp Web, Facebook Messenger (2022), and Instagram Web (2023). Code Verify relies on Cloudflare as a trusted third party as "the source of truth" about the original source of a given application version.

Code Verify checks for specific CSP allowed and disallowed elements, following a policy embedded in the source code, and then computes a SHA-256 hash of a manifest file describing the web application assets. Furthermore, it queries a Cloudflare-owned, sending the product (e.g. WhatsApp) and the version in order to externally obtain the expected hash. The hashes are then compared, and if they match, the version of the application served to the user is considered genuine.

3.6 Isolated Web Apps (2023)

Isolated Web Apps (IWA) [29] are web applications packaged as signed web bundles that include all of their static resources (e.g., HTML, JavaScript, CSS). These bundles are installed through trusted channels, such as app stores or enterprise deployment systems. IWAs are accessed using a custom URL scheme, `isolated-app://`, and are isolated from other web content: the same fixed, strict CSP is always enforced. Each IWA is assigned a unique identity based on its cryptographic signature, and its contents are served from a static archive rather than fetched at runtime. This setup allows browsers to enforce a consistent execution environment for the application.

3.7 Web Source Code Transparency (2023)

Source Code Transparency [14] is a proposal aimed at enabling detection, rather than prevention, of potentially deployed malicious code, with the goal of leaving an audit trail. It would work by sending the hash of the application bundle to a dedicated transparency log, and adding a X509 certificate extension to inform the browser of participation in this mechanism. Due to the how current certificate transparency works, it would be possible for an auditor to detect if a certificate issued for a domain that is expected to have such extension does not have it, signaling an attack. Lastly, the browser would be responsible, once it has computed the web application

³¹<https://www.pageintegrity.net/>

³²<https://developer.mozilla.org/en-US/docs/Glossary/SPA>

³³https://developer.mozilla.org/en-US/docs/Mozilla/Add-ons/WebExtensions/Content_scripts

hash, to fetch from the transparency log an inclusion proof and verify it.

4 THREAT MODEL

The WEBCAT threat model distinguishes between attacks that the system aims to *prevent*, and those it aims to *detect*. The goal is to ensure that all targeted attacks are preventable, and that all large-scale attacks are detectable. An attack is deemed successful if it causes end-user compromise in which users execute client-side code (such as HTML, JavaScript, WebAssembly, or CSS) that was not authored or intended by the application developers. WEBCAT assumes a powerful adversary whose capabilities may extend to include control over infrastructure and over signing identities of application developers. An attacker may compromise a server or take over one or more domains, depending on jurisdiction and technical capabilities. Services, companies, and individual servers are more at risk from attacks, even by small threat actors, than core infrastructure. Known examples include the jabber.ru MITM attack³⁴ and the MyEtherWallet BGP hijacking³⁵, and likely many more cases that went unnoticed—despite being potentially detectable. The distinction between preventable and detectable attacks is a tradeoff between a heuristic estimate of the probability of real-world attacks and the complexity of infrastructure and design.

4.1 Preventable

The system is designed to prevent end-user compromise even if an attacker gains control over:

- *Web server*: Full control over the application server, including domain hijacking, BGP or MITM attacks, rogue TLS certificate issuance, or theft of TLS private keys.
- *OIDC identity*: Up to threshold – 1 developer identities.
- *OIDC issuer*: Up to threshold – 1 identity providers, assuming developers are distributed across issuers.
- *WEBCAT infrastructure*: Including the enrollment, build, and update servers, if there is more than one server for each role and at least one is honest, otherwise it downgrades to detection only.
- *Sigsum log*: Full control of the transparency log.
- *Sigsum witnesses*: Up to quorum – 1 Sigsum witnesses.

4.2 Detectable

Detectability implies the existence of immutable, cryptographically verifiable evidence that an event has occurred, enabling third-party auditing. The system guarantees detectability via auditing and monitoring in the following cases of compromise:

- *Certificate infrastructure*: Unauthorized issuance by Fulcio.
- *OIDC threshold*: Full control of at least threshold identities or identity providers.
- *Extended infrastructure compromise*: Persistent attacks that last longer than the configured *cool-down* period (e.g., 7 days), potentially allowing unauthorized enrollment or policy changes (which would still be detectable, but not prevented).

³⁴<https://notes.valdikss.org.ru/jabber.ru-mitm/>

³⁵<https://www.theverge.com/2018/4/24/17275982/myetherwallet-hack-bgp-dns-hijacking-stolen-ethereum>

Fulcio does not currently support witness cosigning, but such mechanisms could mitigate some signing infrastructure risks and Rekor compromises. Detectability only acts as a deterrent when there are consequences: misbehaving CAs can be quickly removed from trust stores, and broad legal orders can trigger significant disruptions that draw attention. Researchers have previously demonstrated that they could trick Certificate Authorities (CAs) during domain validation to issue rogue certificates [6]. Although Let's Encrypt has deployed mitigations against such attacks, other CAs may not have. BGP hijacking is only one of several possible attack vectors in this scenario.

4.3 Out-of-Scope

The following attacks are explicitly out of scope:

- *WEBCAT code*: Supply chain attacks or backdoors in the WEBCAT client or servers.
- *Application-level issues*: Vulnerabilities or malicious behavior in web applications, even if signed.

5 OVERVIEW

WEBCAT is a framework that includes the mechanisms required to: sign and ship verifiable metadata for browser-based applications; manage enrollment for domains that run these applications; and detect code integrity violations in the end user's browser, blocking content that fails integrity checks before it can execute. The architecture is shown in Figure 3. We identify five categories of actors:

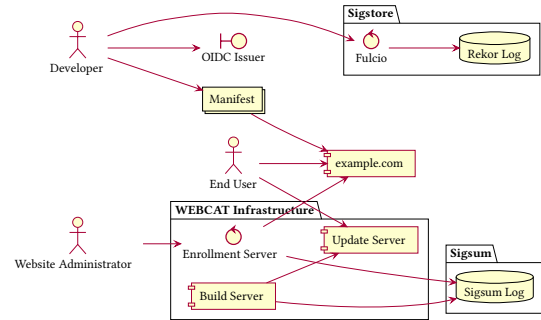


Figure 3: WEBCAT full overview. Arrows are single direction for legibility, and they generally imply a response.

Developers ensure that their applications adhere to a few basic requirements that allow them to run in a secure context, then sign an *application manifest* using Sigstore and one or more OIDC identities.

Infrastructure operators transparently run the enrollment service, which independently and reproducibly build a *trust list* of enrolled domains and their authorized *signing identities*.

Website administrators enroll in a service maintained by infrastructure operators to declare and publicly log the OIDC identities delegated to sign applications for their domain. In the case of centralized applications, such as WhatsApp Web, website administrators and developers are the same entity.

End users use a browser that contains the WEBCAT extension functionality, either natively or via the provided browser extension. When an end user visits a domain, a local lookup determines whether the domain is in the trust list, then performs a series of integrity checks that present a visual success indicator, or block failing components and alert the user.

Auditors and monitors can track OIDC identity issuance, Sigstore artifact signing events, trust list change requests, and trust list builds, and can independently verify artifacts or monitor and alert developers and website administrators in case of anomalous activity.

Developers ensure that all executable assets of their web application, such as JavaScript and CSS, are static. The signed application manifest generated using the WEBCAT signing script contains application metadata and a data structure that includes a cryptographic hash and the corresponding path for executable files that will be loaded directly in the browser. It is signed using Sigstore and one or more OIDC identities, and since signed Sigstore artifacts are transparently logged, once this operation has been completed, it becomes public knowledge.

Centralized web applications combine the developer and website administrator role, as mentioned above, but decentralized applications allow various website administrators to offer deployments of trusted, developer-signed applications. A website administrator indicates their intent to host an instance of an application at a given domain by enrolling their domain via an infrastructure operator-operated enrollment service. The enrollment service then verifies domain ownership and intent to enroll, and waits for a *cool-down* period that prevents unauthorized enrollments or changes, in a process conceptually analogous to the ACME protocol [4]. The enrollment server then signs a payload and publishes the event in a Sigsum [22] transparency log.

Relevant transparency log entries are periodically compiled into a *trust list* by infrastructure operators. The trust list serves two purposes: it determines whether a domain is WEBCAT-enabled, and provides a local copy of the trust material needed to verify the signers for that domain. Similar to the HSTS preload list [10] employed by some Chromium-based browsers and included at build-time, the trust list contains all tuples of the hash of the target domain and the hash of the authorized signers. At every list build event, it is signed with a build key and published in one or more Sigsum logs. This operation can also be outsourced to other semi-trusted operators on independent servers.

Upon startup, an end user's browser checks whether a fresh trust list is available, along with a log inclusion proof to ensure that all updates can be audited. When visiting an enrolled domain, the browser loads the authorized signers and asynchronously downloads the web application manifest, which must be published at the same origin as the application itself. It then verifies the Sigstore signatures in the manifest and ensures that a sufficient number of valid signatures satisfy the policy associated with the domain.

Valid web application manifests—containing either inclusion promises or inclusion proofs for both the Sigstore certificates and artifacts—are cached. At the network level, all requests for potentially-executable content are intercepted, and the content is validated

against the cryptographic hashes in the manifest. CSP also is enforced, and must match the criteria specified in the application manifest. Mismatched response content is discarded and failing component(s) are blocked, ensuring that code that fails integrity checks for a given domain never reaches an execution path; if any integrity checks fail, users are presented with an error message. If all validation succeeds, the end user is presented with a visual success indicator.

The target application functions normally for end-users that do not have the integrity-checking browser components, preserving backward compatibility. For non-enrolled websites, there are no requirements and the performance impact is minimal, as shown in Section 7.1. The system's real-world feasibility is demonstrated via example deployments of well-known open-source applications: Element³⁶, Jitsi³⁷, Globaleaks³⁸, CryptPad³⁹, Bitwarden⁴⁰, and Standard Notes⁴¹. In the evaluation section, we show that no comparable solution exists that addresses both the desired threat model and end-user privacy requirements. Lastly, we present the results of the performance evaluation, showing acceptable slowdowns for enrolled applications and minimal performance impact for everything else.

6 DESIGN

We define a list of requirements informed by several practical criteria. WEBCAT should aim for broad compatibility, and suitability for use with both large and small projects, and open- and closed-source web applications. The system must be feasible in real-world deployments, with minimal development overhead, acceptable runtime performance, and low bootstrap bandwidth and latency costs. Based on these goals, we highlight the following core requirements:

- (1) **Backward compatibility:** The system must not modify or impede access to content for non-enrolled domains, and should introduce no effects for users visiting WEBCAT-enrolled domains without the necessary browser components to perform validation.
- (2) **Threat model coverage:** The system must address both preventable and detectable classes of attacks as outlined in Section 4.
- (3) **Blocking failure mode:** The system must fail closed (i.e., block access when verification fails) in the presence of preventable threats, and must not rely on visual indicators or changes in user behavior to ensure security [1] [27].
- (4) **Trust delegation:** Website administrators must not be required to manage their own signing keys or OIDC identities directly in order to deploy a signed web application, and must be able to delegate trust to developers.
- (5) **Ease of maintenance:** Website administrators must be able to automatically update their applications as soon as new versions are released.
- (6) **Privacy preservation:** The system must not leak user data to third parties; notably, this precludes real-time queries that

³⁶<https://element.io>

³⁷<https://jitsi.org>

³⁸<https://globaleaks.org>

³⁹<https://cryptpad.org>

⁴⁰<https://bitwarden.org>

⁴¹<https://standardnotes.com/>

could divulge a user’s intention to visit a particular domain. Validation must occur either offline or by querying the same origin that serves the application.

- (7) **HTTP-layer distribution:** The user validation process must rely on out-of-band information and data retrievable at the application layer. This decoupling allows the system to function over alternative encrypted transport mechanisms besides HTTPS, such as Tor Onion Services or I2P eepsites.
- (8) **Auditing and monitoring:** The system must be *meaningfully monitorable*, with different actors responsible for observing different parts of the system. This distributes the monitoring load and enables delegated responsibility for specific events.
- (9) **Supported security features:** The system must not require downgrading or departing from modern browser security best practices. Modern web applications employ a variety of mechanisms to enhance internal security, such as using iframe sandboxing⁴² to leverage the Same-Origin Policy (SOP) for isolating keys from untrusted content, or serving more restrictive Content Security Policies (CSPs)⁴³ to workers or frames.
- (10) **Pluggability:** Individual architecture components must be decoupled and readily replaceable, to remain responsive to potential changes in the browser ecosystem, such as browser antitrust policies⁴⁴. Section 9 outlines possible alternatives for several components.

6.1 Developers

Developers must adopt specific practices to enforce a strict separation between data and executable code in their web applications. Executable code, such as JavaScript, HTML, CSS, and WebAssembly resources, must not be dynamically generated by the server, but must be static files. Web applications must also support configuration and deployment without the modification of executable code, instead managing configuration via non-executable assets fetched dynamically at runtime, such as JSON files. Once these conditions are met, developers generate an application manifest that follows the data structure illustrated in Figure 4.

The *Custom Metadata* section of the manifest consists of free-form fields. The version number serves as rollback protection on browsers that support persistent storage. The application manifest must specify one or more Content Security Policies. These are essential components of the application’s security boundary and must be part of the *Integrity Metadata* section, since even minor changes to a CSP can significantly alter an application’s trust and execution model. For applications intended for multiple independent deployments (self-hosting), a CSP is expected to avoid domain-specific components, so that it is deployable without modification by website administrators, since a CSP that requires modification would necessitate a re-signed manifest. See Section 8 for the WEBCAT restrictions on allowed CSP directives; for a discussion of potential CSP extensions that could enhance WEBCAT’s capabilities, refer to Section 9.

⁴²<https://blog.cryptpad.org/2017/08/30/CryptPad-s-new-Secure-Cross-Domain-Iframe/>

⁴³<https://github.com/freedomofpress/webcat/issues/42>

⁴⁴<https://www.w3.org/policies/antitrust-2024/>

Table 1: Signing identities showing domain and issuer diversity. A threshold of 3 out of 4 ensures resilience against compromise of any single domain or identity provider.

Identity	Domain	Issuer
alice@example.com	example.com	Google
bob@example.net	example.net	Google
carol@example.com	example.com	GitHub
frank@example.net	example.net	Microsoft

Finally, developers must publish a signing policy, and sign the manifest in accordance with this policy. A signing policy consists of a list of tuples, each containing an identity and its corresponding issuer, along with a threshold parameter. Developers are encouraged to require more than one signature (i.e., threshold > 1) and to diversify both identities and issuers, using at least two distinct identity providers and, ideally, identities from different domains. Table 1 provides an example of a signing policy that meets these criteria, highlighting issuer and domain diversity and reducing the risk of a single point of compromise. The signing threshold should be set lower than the total number of identities, to maintain signing capabilities resilient to personnel changes or loss of access to a signing identity. Publishing the application’s signing policy allows website administrators hosting an instance of the application to provide a matching policy on trusted signers upon enrolling their domain, which is the recommended configuration.

Signing can be performed using any Sigstore client, according to Figure 1. Currently, the *Public Good* instance is used, but a dedicated WEBCAT Sigstore infrastructure deployment is also fully supported. Conceptually, Sigstore could also be replaced with SCITT⁴⁵, should it reach standardization, or even with traditional cryptographic key-based signing, for example backed by hardware security modules.

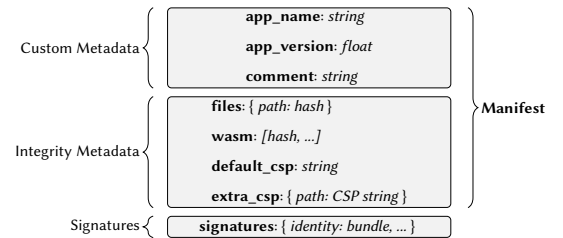


Figure 4: Manifest data structure.

6.2 Website Administrators

Website Administrators preparing to serve a signed web application must select a dedicated domain or subdomain on which to host it. Since browser context isolation relies on the Same-Origin Policy, mixing signed and unsigned executable content would break WEBCAT’s underlying security assumptions, and therefore is not permitted.

Administrators must then decide whether to adopt the application’s published signing policy or define a custom policy. Using

⁴⁵<https://datatracker.ietf.org/group/scitt/about/>

the application’s original policy is generally recommended; this allows updates to be applied automatically without administrative intervention, and avoids requiring website administrators to participate in a signing process. Adopting and advertising the developer’s original signing policy also serves as a signal of trustworthiness to users, who are assured that they are receiving the same application as users of other hosted instances. That said, customized signing policy definitions are supported, for instance, to strengthen an application’s existing policy by enforcing additional constraints, such as requiring valid signatures both from the developers and from the website administrators themselves, or to allow administrators to make code-level modifications and assume independent signing responsibility. To deploy an application and enroll it into the system, administrators must:

- (1) Assign a domain or subdomain to define the application’s scope.
- (2) Deploy the web application assets without modification.
- (3) Ensure that all Content Security Policies specified in the application’s manifest are applied.
- (4) Add HTTP headers that declare the signing policy.
- (5) Add HTTP headers indicating the intention to participate in WEBCAT.
- (6) Upon completion of the above, submit the domain to the enrollment service.

Administrators must be able to update or revoke the signing policy associated with their own domains, for example due to a desire to discontinue hosting the signed application or in the event of a domain ownership change. However, they must also be protected from unauthorized attempts to alter the enrollment status of their domains, such as hostile takeovers, denial-of-service attacks, and accidental enrollment or de-enrollment, reminiscent of the types of security and usability issues that contributed to the deprecation of HPKP [32]. As discussed in the next section, the introduction of a *cool-down* period provides a detection and intervention buffer, mitigating the risk of hostile takeovers via spurious enrollment requests. Similarly, de-enrollment requests are initiated and finalized only after the respective cool-down periods have elapsed.

Signing policy updates are generally infrequent events; in the case of a stable project, a signing policy may remain unchanged for years. Further, the use of OIDC identities for signing—or more generally, offline keys rather than server-accessible keys—significantly reduces the need for frequent rotation and updates. Signing policy updates are therefore higher-salience data points amenable to increased scrutiny.

Changes to a signing policy, however, introduce additional complexity. During the transition from one signing policy to an updated one, some users may have already updated their trust list, while others may still rely on the older version. To address this inconsistency, manifests support incremental signing, where a previously-signed manifest can be co-signed with new identities, even if the original identities are no longer available. This allows the manifest to remain valid under both the old and the updated policies. Servers then publish backup policy headers, enabling clients to recognize both the new and old policies.

6.3 Enrollment Server

The enrollment server is part of the WEBCAT infrastructure and is responsible for ingesting and processing enrollment requests submitted by website administrators. The server does not enforce authentication, and anyone can submit a request for any domain, but the enrollment server independently verifies control over the target domain. This mirrors the logic of certificate issuance in the ACME protocol: anyone can initiate a certificate request, but only the rightful domain owner can successfully complete the verification challenge [4].

The enrollment server is designed as a state machine, with an asymmetric keypair and performs basic validation as described in Algorithm 4. The first validation step is checking if the same domain is already in the processing queue, in order to avoid race conditions; then it is checked whether the requested change is compatible with the current list state: a domain can be added only if it is not already present, while a modification or a deletion can be performed only in the opposite case. Once verified, the change request is logged into Sigsum once (using the enrollment server keypair). After the cool-down period, the verification is performed again, and in case of success, the change is confirmed by sending a payload containing the confirmation information to Sigsum. The change is then committed to the internal state of the enrollment server. An RFC 3161-compliant Time-Stamp Authority (TSA) [2] must be used by the enrollment server to cryptographically prove that it respected the required cool-down period. The signed timestamp should be embedded in the payload and made available as needed. While strict auditing could, in principle, detect forged timestamps, relying on a trusted third party allows automated and verifiable enforcement of timing constraints during the list build process.

In traditional transparency systems, such as Certificate Transparency or other Trillian-based logs⁴⁶, metadata is typically served directly by the log infrastructure. In contrast, Sigsum decouples metadata storage from the log itself, allowing for reduced hosting costs and the use of general-purpose infrastructure. Since Sigsum only logs checksums and signatures, WEBCAT’s transparency requirements are fulfilled by providing access to the metadata outside of Sigsum. The enrollment server must therefore serve all historical payloads associated with its Sigsum submissions, for example via an API. The enrollment server constitutes a single point of failure: it could forge enrollment requests or fabricate the results of header retrieval from target domains. However, even in the case of a malicious or compromised enrollment server, the system design enforces a mandatory cool-down period before any change becomes effective. This delay provides a critical window for detecting and investigating suspicious activity; if a malicious modification is identified during this time, it can be prevented from being finalized and merged.

Furthermore, while out of scope for this research, the enrollment server could be replaced by a distributed architecture, such as multiple servers running a consensus protocol [9], or even a permissioned blockchain [31]. This would align with the broader goal of WEBCAT to support a multi-browser, multi-platform ecosystem,

⁴⁶Trillian is Google’s open-source log implementation, used among others, by Rekor. See <https://github.com/google/trillian>

Algorithm 4 WEBCAT.Infra.Enroll

Require: Submitted domain d , Cool-down delta δ

```

1: procedure ENROLL( $d$ )
2:   if ISINQUEUE( $d$ ) then
3:     return ERR("domain already being processed")
4:   end if
5:    $headers \leftarrow$  FETCH( $d$ )
6:   if  $\neg$ ISACTIONVALID( $headers, d$ ) then
7:     return ERR("invalid or inconsistent action header")
8:   end if
9:   if  $\neg$ VALIDATEPOLICY( $headers$ ) then
10:    return ERR("policy header invalid or unchanged")
11:  end if
12:   $payload_{initial} \leftarrow$  CREATEPAYLOAD( $ts, d, headers, 'initial'$ )
13:   $ts \leftarrow$  TIMESTAMPFROMTSA( $H(payload_{initial})$ )
14:   $signed_{initial} \leftarrow$  SIGN( $payload_{initial}$ )
15:  SIGSUMSUBMIT( $signed_{initial}$ )
16:  WAIT( $\delta$ )
17:   $headers_{final} \leftarrow$  FETCH( $d$ )
18:  if  $headers \neq headers_{final}$  then
19:    return ERR("policy changed during cool-down period")
20:  end if
21:   $ts \leftarrow$  TIMESTAMPFROMTSA( $H(payload_{final})$ )
22:   $payload_{final} \leftarrow$  CREATEPAYLOAD( $ts, d, headers, 'final'$ )
23:   $signed_{final} \leftarrow$  SIGN( $payload_{final}$ )
24:  SIGSUMSUBMIT( $signed_{final}$ )
25:  UPDATALISTSTATE( $d, headers$ )
26:  return OK
27: end procedure

```

where trust must be distributed both technically and organizationally.

6.4 Build Server

One or more build servers are responsible for building the trust list. While the enrollment server could theoretically perform this task, a separation of responsibilities increases WEBCAT's resistance to infrastructure compromise: in a single-server enrollment server model, if the enrollment server also built the trust list, it could readily fabricate forged versions of the list. Instead, the build process is run independently, for example, as a pre-scheduled job, rather than an internet-facing service. It retrieves all signed events submitted using the enrollment server's keys from Sigsum, retrieves the corresponding metadata via the enrollment server's API, and confirms an exact match. It then verifies each event, checks that the required cool-down period was observed, and reconstructs the trust list deterministically. Because the trust list output is bit-for-bit reproducible, any misbehavior can be detected by comparing results across multiple independently-run builds. Although any single build server could theoretically forge an update, this risk is mitigated by the reproducibility of the trust list, as well as optional co-signing by multiple build servers. The build process itself is stateless and isolated from external exposure: it requires no persistent storage or network availability, aside from read access to public logs and metadata, and write access to a distribution target (e.g.,

object storage). The build server's only sensitive asset is its signing key, which should reside in hardware-backed key storage.

Algorithm 5 provides a high-level summary of the process. The built list consists of a binary file containing all tuples of the form $(H(domain), H(policy))$, where H is a collision-resistant hashing function, assumed to be SHA-256. Tuples are written in raw binary format without separators or serialization. Since the output size of SHA-256 is fixed, each enrolled domain contributes exactly 64 bytes to the list. As a result, the list size scales linearly, reaching approximately 6.4 MB for 100,000 enrolled domains. Given that list changes are expected to be infrequent, future work could explore support for incremental updates. See Section 9 for further proposals on scalability concerns.

6.5 Update Server

The update server provides trust list updates to end users and preserves the availability of all past updates for auditing purposes. One or more build servers must be able to push updates to it. It is the component in the system that requires the highest availability and bandwidth guarantees, as it must serve downloads to potentially millions of users per day. Being a low-value target, it can be easily distributed across CDNs; the update server can be any accessible storage backend, such as object storage.

6.6 End Users

End users, who are the primary beneficiaries of the system, require a built-in browser component. During its startup phase, this component updates the Sigstore trust material via TUF and fetches the latest list update metadata. The update metadata is then verified using Sigsum and, if valid, the update file is fetched and applied locally, as illustrated by Algorithm 6.

This update procedure should run at browser startup and, for long-lived processes, at regular intervals matching WEBCAT's update schedule. For example, if list updates are built and published daily at midnight, clients should check for updates at every startup—provided their last update is older than the latest build—and additionally once per day, if they remain active. The Sigsum tree size serves as a simple yet effective mechanism to detect rollback attacks, and this design choice was made for its simplicity. However, the list update mechanism is also fit for TUF, which provides stronger guarantees against partial key compromise, improved freshness semantics, and rollback and freeze protections.

At every resource load across all tabs, Algorithm 7 is executed. To minimize performance overhead, especially for domains not enrolled in the system, aggressive caching is used. Backward compatibility is preserved, as clients without WEBCAT browser functionality simply ignore the additional headers and the manifest. In contrast, a client that does support WEBCAT performs both integrity and transparency verification. The verification is blocking: if there is a CSP or content mismatch, or if the manifest does not meet the required threshold of signatures, the page load is aborted at the network level. As previously noted, manifests may include more signatures than required, which is particularly useful during policy transitions; any extra signatures are safely ignored. While the use of cryptographic verification and network-level request interception may appear costly, Section 7 demonstrates that the

Algorithm 5 WEBCAT.Infra.BuildList

Require: Log public key $logpk$, Enrollment server public key pk , Witness public keys Σ_{pk} , Quorum q

```

1: procedure BUILDLIST
2:    $db \leftarrow \text{NEWDATABASE}()$ 
3:    $sth \leftarrow \text{GETSTH}()$ 
4:   for  $i \leftarrow 0$  to  $sth.size - 1$  do
5:      $leaf \leftarrow \text{GETLEAF}(i)$ 
6:      $(keyHash, sig, checksum) \leftarrow leaf$ 
7:     if  $keyHash \neq H(pk)$  then
8:       continue
9:     end if
10:     $(payload, proof) \leftarrow \text{FETCH}(checksum)$ 
11:     $(sth, leaf, inc, \Sigma_w) \leftarrow proof$ 
12:    if  $leaf.checksum \neq checksum$  then
13:      return  $\text{ERR}(\text{"payload does not match Sigsum entry"})$ 
14:    end if
15:    if  $\neg \text{SIGSUM.VERIFY}(payload, sig, pk, logpk, inc, sth, \Sigma_w, \Sigma_{pk}, q)$ 
then
16:      return  $\text{ERR}(\text{"invalid Sigsum proof for leaf i"})$ 
17:    end if
18:     $\text{COMMIT}(db, payload)$ 
19:  end for
20:  for all  $payload \in db$  do
21:    if  $\neg \text{VERIFYTSA}(payload.ts, H(payload))$  then
22:      return  $\text{ERR}(\text{"invalid timestamp"})$ 
23:    end if
24:    if  $payload.type = \text{"initial"}$  then
25:      if  $\neg \text{ISACTIONVALID}(payload.headers,$ 
 $payload.domain)$  then
26:        return  $\text{ERR}(\text{"action is invalid for the domain"})$ 
27:      end if
28:       $initials[payload.domain] \leftarrow payload$ 
29:    else if  $payload.type = \text{"final"}$  then
30:      if  $payload.domain \notin initials$  then
31:        return  $\text{ERR}(\text{"missing initial payload"})$ 
32:      end if
33:       $initial\_payload \leftarrow initials[payload.domain]$ 
34:      if  $initial\_payload.headers \neq payload.headers$ 
then
35:        return  $\text{ERR}(\text{"headers changed between initial$ 
 $\text{and final payload"})$ 
36:      end if
37:      if  $(payload.ts - initial\_payload.ts) < \delta$  then
38:        return  $\text{ERR}(\text{"cool-down period not respected"})$ 
39:      end if
40:    end if
41:  end for
42:   $list \leftarrow \text{BUILD}(db)$ 
43:   $signed \leftarrow \text{SIGN}(list)$ 
44:   $proof \leftarrow \text{SIGSUMSUBMIT}(signed)$ 
45:   $\text{PUSHUPDATE}(list, proof)$ 
46:  return  $\text{OK}$ 
47: end procedure

```

Algorithm 6 WEBCAT.Client.Update

Require: Trusted TUF root $root$, Previous tree size t_{prev} , Previous list hash $hash_{prev}$, Log public key $logpk$, Witness public keys Σ_{pk} , Quorum q , Build server public key $buildpk$, Current time ts

```

1: procedure UPDATE
2:    $ca, rekor.pk \leftarrow \text{TUF.UPDATETARGET}(\text{"sigstore"}, root, ts)$ 
3:    $(hash, proof) \leftarrow \text{FETCHLISTUPDATE}$ 
4:    $(sth, leaf, inc, \Sigma_w) \leftarrow proof$ 
5:   if  $\neg \text{SIGSUM.VERIFY}(list, sig, buildpk, logpk, inc, sth, \Sigma_w, \Sigma_{pk}, q)$ 
then
6:     return  $\text{ERR}(\text{"invalid Sigsum proof"})$ 
7:   end if
8:   if  $hash = hash_{prev}$  then
9:     return
10:  end if
11:  if  $sth.size \leq t_{prev}$  then
12:    return  $\text{ERR}(\text{"potential rollback attack"})$ 
13:  end if
14:   $list \leftarrow \text{FETCH}$ 
15:  if  $hash \neq H(list)$  then
16:    Return  $\text{ERR}(\text{"list does not match metadata"})$ 
17:  end if
18:   $\text{IMPORTLIST}(list)$ 
19:   $hash_{prev} \leftarrow hash$ 
20:   $t_{prev} \leftarrow sth.size$ 
21: end procedure

```

overhead is negligible for non-participating domains and minimal for participating domains for real-world applications.

6.7 Auditors and Monitors

The architecture enables a defined separation of auditing⁴⁷ and monitoring⁴⁸ responsibilities, which helps reduce cognitive burden and mitigate alert fatigue [3]. Developers are expected to monitor the issuance of Fulcio certificates associated with any OIDC identities included in their advertised signing policy. Alerts should only be triggered during signing events—typically aligned with release operations—making manual reconciliation straightforward and infrequent.

Both developers and the broader security community (particularly in the case of open-source projects) should be able to automatically retrieve all Rekord log entries that satisfy a given project's signing policy and correlate them with actual release artifacts. Website administrators, in turn, are only responsible for detecting and validating list policy changes targeting their own domain.

At the ecosystem level, independent entities and security researchers should operate Sigstore and Sigsum witnesses to ensure log consistency, detect misbehavior, and prevent split-view attacks, thus auditing the system. They should also monitor WEBCAT list updates, independently reconstruct the trust list history, and verify its integrity over time. Table 2 summarizes the expected auditors,

⁴⁷An auditor is responsible for verifying the consistency or other integrity properties of a transparency log and detecting misbehavior.

⁴⁸A monitor is responsible for processing the log content, and alerting potentially interested parties in case of certain events.

Algorithm 7 WEBCAT.Client.Verify

Require: Origin *domain*, HTTP response *response*, Internal trust list *T*, Current time *ts*

```

1: procedure VERIFYRESOURCE(domain, response)
2:   if type  $\notin$  [HTML, JS, CSS, WASM] then
3:     return RESPONSE
4:   end if
5:   policyHash  $\leftarrow$  FETCH(T, domain)
6:   if  $\neg$ policyHash then
7:     return RESPONSE
8:   end if
9:   manifest  $\leftarrow$  FETCH(domain, "webcat.json")
10:  (headers, body)  $\leftarrow$  response
11:  (signers, threshold)  $\leftarrow$  headers.policy
12:  if polichHash  $\neq$  H(signers, threshold) then
13:    return ERR("server returned invalid policy")
14:  end if
15:  valid  $\leftarrow$  0
16:  for all bundle  $\in$  manifest.signatures do
17:    if bundle.identity  $\in$  policy.signers then
18:      if SIGSTORE.VERIFY(bundle.signature, manifest,
19:                          bundle.cert, bundle.proof)
20:    then
21:      valid  $\leftarrow$  valid + 1
22:    end if
23:  end for
24:  if valid < threshold then
25:    return ERR("signatures do not meet threshold")
26:  end if
27:  if headers.csp  $\notin$  manifest then
28:    return ERR("CSP does not match manifest")
29:  end if
30:  if H(body)  $\notin$  manifest then
31:    return ERR("response body does not match manifest")
32:  end if
33:  return RESPONSE
34: end procedure
    
```

the components they are responsible for, and the typical frequency of alerts, highlighting in particular that website administrators would receive only relevant notifications, likely fewer than one per year.

7 EVALUATION

Evaluation is conducted across several criteria: impact on end-user performance, coverage of the threat model and design requirements, comparison with related work, and compatibility with existing web applications.

7.1 Performance

The main performance metric evaluated is the impact of the browser component on end users. The implementation is written in TypeScript and leverages the WebExtensions and Web Cryptography

APIs. It also includes several workarounds to overcome inherent limitations of these browser mechanisms. To test performance, we considered two scenarios: **cold** and **warm** conditions. In cold conditions, a fresh Firefox profile is launched for each test run. The performance case is executed exactly once per session, aiming to measure the overhead on the *first load* of a website, with all possible browser caches empty. In warm conditions, the target URL is first loaded once to populate the cache. The measurement is then taken on the *second load*, reflecting typical usage where some assets are cached. For each scenario, we examined three test conditions:

- (1) **Baseline:** the browser component is disabled entirely.
- (2) **Non-enrolled domain:** the browser component is enabled, but the target domain is not part of the trust list.
- (3) **Enrolled domain:** the browser component is enabled, and the domain is on the trust list.

All test conditions were executed in two environments: locally, with the domains pointing to a local web server, to minimize network variability and isolate the impact of the browser component itself; and remotely, with the domains pointing to a live server, to evaluate performance under real-world network conditions. To ensure comparability, the same server is used in all cases. Two domain names point to this server: one included in the trust list and one excluded, ensuring consistent server-side performance across test conditions. As a test case, we chose Element, an open-source web application⁴⁹. Element is a complex web application that loads numerous assets, including WebAssembly modules, providing a realistic workload.

When conducting browser-based testing, confounding factors can include JIT engine garbage collection events, network fluctuations, and operating system scheduling. To mitigate test variability introduced by these effects, tests were conducted using a clean Firefox environment for each repetition on a dedicated server running only one benchmark at a time.

Figure 5 shows the local performance results. Locally, the overhead for non-enrolled domains is below 2% under cold conditions and below 5% under warm conditions. Remote test results exhibited too much network fluctuation and variability to meaningfully observe the impact of the extension. In scenarios where the network connection was stable and reliable, such as datacenter-to-datacenter connections within the same geographic area, the results closely matched those obtained locally. However, in cases involving higher latency and variability, such as cross-continent links or typical home DSL connections, the results were too scattered to draw generalizable conclusions, even when increasing the number of repetitions.

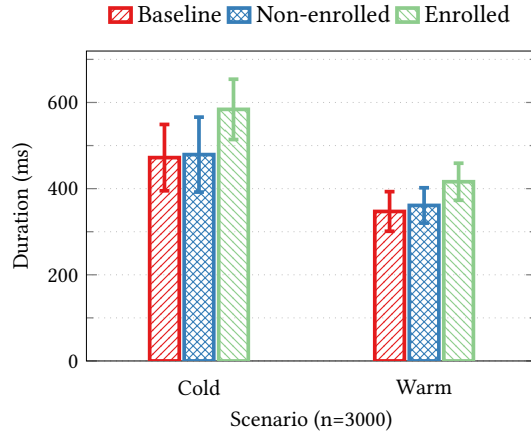
7.2 Compatibility with Existing Web Applications

A selection of commonly self-hosted applications was chosen to test for compatibility, in order to assess the system's real-world usability. The following steps are necessary to evaluate and integrate an existing web application, and approximate the workflow of a developer interested in adopting the system for their application:

⁴⁹<https://element.io/>; see also 7.2.1

Table 2: Auditing and monitoring roles and responsibilities. Responsibilities are distributed to minimize cognitive burden and ensure meaningful alerts.

Actor	Target	Log	Data source	Alert frequency
Developers	Fulcio certificate issuance	Rekor	Rekor API	At every release
Developers/Ecosystem	Combined signing events	Rekor	Rekor API, Releases	At every release
Website Administrators/Advanced Users	Policy changes	Sigsum	Enrollment server	Rare
Ecosystem	Log consistency (Witnesses)	Sigsum, Rekor	N/A	Never
Ecosystem	List updates	Sigsum	Update and Enrollment servers	Never

**Figure 5: Unified plot of all local tests, grouped by scenario.**

- (1) Investigate compatibility: the application must not have server-generated scripts, and must not rely on the dynamic evaluation of code or other similar features.
- (2) Define a secure Content Security Policy: in order to be accepted and verified by end-users, the CSP must satisfy certain specified requirements. It is essential in this phase to understand if there are minimal application changes that can be performed to ensure compatibility. Lastly, it is still a useful exercise to try to obtain the strictest working CSP possible.
- (3) Complete any necessary code changes as identified during the previous two phases, such as moving inline scripts to external files.
- (4) Create a configuration file according to the format specified in the WEBCAT project documentation. The configuration file includes the application name, version, default CSP, and any additional CSPs that may be needed for specific paths. Include any WebAssembly blob hashes inlined in the JavaScript code, if applicable.
- (5) Run the signing script, signing with as many OIDC identities as needed.

Once a web application’s initial compatibility has been confirmed, the above procedure can be fully automated within any continuous integration pipeline. If no code changes are required, the CSP is predetermined, and any blob hashes are known, the WEBCAT configuration file can be prepared in advance and included

in the source code, and after a build process completes, a signing script can automatically populate any remaining fields. Sigstore already fully supports automated signatures via pre-issued tokens, so once compatibility is established, no significant manual overhead is required. Website administrators, namely those hosting the web application and delivering it to their community, are only responsible for adding the necessary headers for inclusion in the list, along with any required CSP headers, and for deploying the application on their web server. They are not responsible for signing the application and cannot modify the code themselves, since the trusted identities are those of the developers. Future updates can be deployed non-interactively using standard automation tools, enabling unattended upgrades. Manual intervention is only needed if the developers announce changes to the signing identities.

7.2.1 Element. Element is a client for the Matrix decentralized messaging protocol.⁵⁰ It is packaged for a variety of platforms: a web-based version, and mobile and desktop applications. It is a fully static application that needs a backend service to communicate with the underlying Matrix server. The following procedure was tested on version 1.11.94, distributed from the official Debian package repository. With the official Element deployment⁵¹ as a reference, we can formulate a strict enough CSP that satisfies both our requirements and the application’s, and build a WEBCAT configuration file with descriptive metadata and our chosen CSP. The configuration file can then be used as an input for the signing script, which will proceed automatically. Website administrators have to deploy the web application assets, the signed manifest and all the required headers. In our demo setup, we deployed everything on Cloudflare Pages with minimal effort. In the test version, we are still using the `element.io` backend services, while in a real self-hosting setup, the configuration would point to a different backend. This use-case is supported by our proposed architecture, since the configuration file is in a separate JSON-formatted file, that is fetched via a `fetch()` call and then parsed, and not sourced as script. Since it is not executable, and does not need to be signed, website administrators can change the configuration without affecting the application’s code integrity and needing new signatures, a requirement for supporting decentralized, self-hosted applications.

7.2.2 GlobaLeaks. GlobaLeaks (5.0.44) is one of the more widespread open-source whistleblowing platforms. It is used both by newspapers and companies alike, and is even used for compliance

⁵⁰<https://spec.matrix.org/latest/>

⁵¹<https://element.io/>

with the EU Whistleblower Protection Directive⁵². A WEBCAT-compatible GlobaLeaks instance did not require any code modification; manifest building required collecting WebAssembly blobs and an additional CSP that the application uses to enforce some restrictions on its iframe sandboxing and service workers⁵³.

7.2.3 CryptPad. CryptPad⁵⁴ is an open-source encrypted collaboration suite that includes editing software for documents, spreadsheets, slides, and diagrams. As of version 2024.12, CryptPad includes some inline scripts that have to be moved to dedicated files to comply with WEBCAT's integrity requirements. Additionally, the application loads a per-instance configuration via server-generated scripts, which are sourced dynamically through an API. For demonstration purposes, we collected the hash of these server-generated scripts and included them in the manifest. However, in a production-signed deployment, it would be preferable for configuration-loading to be handled by a static file such as a JSON file. This would avoid the need for signing instance-specific configuration information, and make a WEBCAT-compatible CryptPad deployment host agnostic.

CryptPad also employs iframe sandboxing on a subdomain, using the Same-Origin Policy (SOP) to isolate cryptographic key handling from areas where user-generated content is processed. This is a secure and effective practice, also adopted by GlobaLeaks, although GlobaLeaks uses the same domain with a different CSP for the iframe path. However, this practice introduces portability challenges, since the domain allowed for each iframe must be embedded in the signed application manifest. While this is feasible from a technical standpoint, it undermines the "sign once, deploy many times" model, since it means a manifest must be domain-specific. CryptPad could consider adopting GlobaLeaks' approach, although this may weaken SOP guarantees. Alternatively, the CSP standard could be extended to allow subdomain wildcards without specifying full FQDNs, for example by allowing `frame-src: sandbox.` to imply `frame-src: sandbox.example.com`. Future iterations of WEBCAT could also support this dynamically, resolving such constraints at runtime in place of the browser.

7.2.4 Jitsi. Jitsi is an open-source video conferencing application.⁵⁵ As of version 2.0.10008, Jitsi uses Server Side Includes (SSI) as a templating mechanism, which prevents integrity hashes from being pre-generated from the official package. We developed a simple Python script that pre-renders the SSI into static HTML content and moves inline scripts to dedicated files. Though this runs contrary to the original purpose of using SSI, our understanding is that Jitsi primarily uses SSI for per-instance personalization. For optimal WEBCAT compatibility, we encourage branding to rely on non-executable assets where possible, or to be performed dynamically using `document.write` as a fallback.

7.2.5 Bitwarden. Bitwarden⁵⁶, a multi-platform open-source password manager, does not require any code-level changes for WEBCAT compatibility as of version 2024.12.1.

7.2.6 Standard Notes. Standard Notes⁵⁷, an open-source encrypted collaborative notes app, does not require any code-level changes for WEBCAT compatibility as of version 3.195.13.

8 IMPLEMENTATION

The full WEBCAT architecture has been implemented⁵⁸, with the end-user browser functionality packaged as a browser extension due to the current lack of native browser support; missing browser features required for a production-grade deployment are discussed below. Design choices aimed at providing support for a wide range of existing web applications, as well as balancing performance and functionality, are also discussed.

8.1 Cryptography, Performance, and Extension Design

Every cryptographic operation in WEBCAT is implemented using the Web Cryptography API, and is performed asynchronously whenever possible. As noted previously, Ed25519 is not yet standardized in the Web Cryptography API, there is a community proposal to standardize it⁵⁹, and it is supported by all major browsers. Ed25519, ECDSA, RSA and SHA-256 are sufficient for all cryptographic operations required by Sigsum, Sigstore, and TUF. Nonetheless, extensive wrappers were necessary to support certificates, proofs, checkpoints, and similar data formats. A browser-native implementation of all three systems has been built from scratch, with no runtime dependencies, in order to optimize performance and keep the extension footprint as small as possible.

The Content Security Policy is verified on every request, based on the path and the directives specified in the manifest. At runtime, CSP headers are also parsed and explicitly checked for disallowed directives such as `unsafe-eval` or `unsafe-inline`. File checksums are verified at the network level before responses are passed to the DOM engine, ensuring that untrusted resources can be definitively dropped before execution. Only requests and responses with types potentially associated with executable material are intercepted⁶⁰, leaving most web traffic and user-generated content unaffected, and avoiding unnecessary performance impacts. Manifest and list data are heavily cached. Local list lookups are performed using IndexedDB, allowing for fast lookups even with hundreds of thousands of entries. Many operations are implemented asynchronously and awaited only when the first byte of a response arrives—only when strictly necessary.

WEBCAT avoids relying on Subresource Integrity. There are multiple reasons for this choice: one is that the WebExtensions API does not expose whether a resource is being loaded with SRI or not, thus requiring DOM parsing to discover that, as WAIT does. Another one is that SRI currently does not support scripts loaded from other scripts, such as ES6 module imports or service workers⁶¹.

⁵⁷<https://standardnotes.com/>

⁵⁸<https://github.com/freedomofpress/webcat?tab=readme-ov-file#web-based-code-assurance-and-transparency-webcat>

⁵⁹<https://wicg.github.io/webcrypto-secure-curves/>

⁶⁰While MIME types are spoofable, the extension uses the `ResourceType` variable set by the browser when the request is initiated. The type is then inferred by the origin, whether it was called from a `img src` or a `script src`, avoiding any reliance on the server-provided types. See: <https://developer.mozilla.org/en-US/docs/Mozilla/Add-ons/WebExtensions/API/webRequest/ResourceType>

⁶¹<https://github.com/w3c/ServiceWorker/issues/1680>

⁵²Directive (EU) 2019/1937 of the European Parliament and of the Council of 23 October 2019 on the protection of persons who report breaches of Union law.

⁵³<https://github.com/freedomofpress/webcat/issues/42>

⁵⁴<https://cryptpad.org/>

⁵⁵<https://jitsi.org/>

⁵⁶<http://bitwarden.com/>

A WAIT-like approach requiring DOM parsing and communication with a background script would open up an entire class of potential security vulnerabilities.

8.2 Integrity Enforcement and Runtime Controls

Network-level integrity checks apply to all JavaScript, CSS, and HTML files, as well as to service workers, web workers, and shared workers. Enrolled origins can embed iframes from other enrolled origins, and in these cases, CSP is dynamically parsed to block any mixed content, preserving security boundaries. This approach supports complex web applications like CryptPad, which use iframe sandboxing⁶², without requiring them to reduce their own security posture, as discussed in Section 7.2.

WebAssembly is a special case. WebAssembly blobs are fetched via XHR or fetch() and are indistinguishable from non-executable content such as images or user-generated files; the only CSP directive that affects WebAssembly is `wasm-unsafe-eval`. As a result, WebAssembly enforcement happens at runtime: a hooking script is injected into every JavaScript asset (at the network level) of an enrolled origin, along with a list of allowed hashes. Integrity is checked by overwriting the original WebAssembly methods with authenticated wrappers and then freezing them⁶³. The hook also verifies whether any leftover service worker, installed before enrollment, could intercept requests, and if so, reloads it with full integrity checks in place. As a consequence, unsigned WebAssembly is prevented from reaching the compile phase entirely, which is also often abused by JIT exploits⁶⁴.

9 DISCUSSION

There are both clear limitations and areas that require further research within this architecture. Some of these could be relatively easy to overcome and are acknowledged here as decisions made to keep the scope of this project contained. Others are implementation-specific, constrained by the technical choices made for the proof-of-concept, such as building the component as a browser extension rather than a built-in browser patch. This decision was made partly to allow a beta testing phase and easier distribution to interested users, but also to reduce prototyping complexity. As a result, the extension is naturally limited by the available WebExtension APIs, compared to the broader access and flexibility a native browser integration would offer.

A real-world deployment would ideally involve full browser integration, or at the very least, support for a more fine-grained CSP, especially with regard to WebAssembly, which can currently only be allowed via the `wasm-unsafe-eval` directive. Additionally, Chrome compatibility is currently not implemented nor planned, and may never be feasible given the deprecation of Manifest V2 APIs, which removes the ability for extensions to perform blocking network intercepts. As mentioned earlier, the current list update

mechanism could be replaced by TUF to gain better freshness guarantees, rollback protection, and resilience against key compromise. Sigstore, too, while functional and available today, is considered controversial in parts of the academic community: it uses a range of file formats for signatures and proofs, and has a centralized architecture. That said, the original design emphasized pluggability precisely to allow for systems like this to be swapped out or complemented with other alternatives. Beyond these technical implementation details, there are also broader, architectural questions that should be addressed before a production-ready system can be released, which are discussed in the following subsections.

9.1 List Scalability

The first and most evident limitation is scalability of the trust list. Since the trust list grows linearly with the number of enrolled domains, a list of 1 million entries would result in a size of 64 MB, possibly excessive for this kind of functionality. On this topic, several observations can be made: as mentioned earlier, while TLS is designed for the entire web, WEBCAT targets a specific subset of client-side applications or cases where developers or website administrators want to assert the provenance of their pages, such as when deploying static content to a CDN, and we can expect list sizes to remain relatively small. Automated maintenance actions could help manage list size: for example, scheduling the removal of expired domains, or periodically rechecking hostnames that no longer resolve. Any such pruning actions must still respect our cool-down period, but they could help avoid a scenario where the list only grows without bounds.

As mentioned in Section 5, the trust list serves a dual purpose, providing a local copy of both the list of WEBCAT-enrolled domains and the trust material needed to verify the signers for a given domain. If the list were used simply to determine enrollment, list entry sizes could be drastically reduced. For TLS-enabled websites, it might be possible to use a similar approach to CRLite [18], first applying cascading bloom filters, then an inverse filter, to construct a compact representation of enrolled domains without including any trust material; this alone would prevent stripping attacks. The actual trust material could then be distributed by the website itself, through a mechanism somewhere between OCSP stapling and providing cryptographic proofs from a CONIKS-like system [20].

9.2 List Spam

There are also potential spam issues with the trust list. Currently, a single entity with control over a given domain could submit an unlimited number of entries very cheaply, for instance by using a wildcard to serve all requests. Another spam vector involves Tor's .onion domains. Onion domains have a fundamentally different ownership model than traditional clearnet domains: they are free and unlimited to create, and .onion domain identifiers are directly tied to cryptographic keys rather than registrars. They are not transferred or sold in the traditional domain registration process, as ownership is bound to the private key itself. It is also well known that threat actors with adversarial interests toward the Tor network have incentives to degrade its performance and services [15].

Spam mitigation is a complex topic. Potential solutions generally enforce a form of economic cost or friction. Solutions range from

⁶²<https://blog.cryptpad.org/2017/08/30/CryptPad-s-new-Secure-Cross-Domain-Iframe/>

⁶³https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Object/freeze

⁶⁴<https://www.zerodayinitiative.com/blog/2024/5/2/cve-2024-2887-a-pwn2own-winning-bug-in-google-chrome>

mechanisms employed generally, such as CAPTCHAs and proof-of-work mechanisms, to WEBCAT-specific mitigations, such as setting hard enrollment limits on the number of subdomains per top-level domain.

9.3 Malicious Enrollment

Particular care must be taken to avoid permanent Denial of Service situations, even self-inflicted ones, which were a key reason behind the deprecation of HPKP [32]. At a higher level, WEBCAT differs radically from HPKP, both in functionality and in the associated risks. In HPKP, the end user was responsible for recording and persisting the pinned material, with no out-of-band update mechanism: this limitation contributed to the specification's eventual withdrawal. By contrast, WEBCAT provides an out-of-band update mechanism via the list and its update system, making it no more vulnerable to this class of issues than any other software that relies on regular updates. Similarly, the enrollment service in WEBCAT does not rely on verifying prior trust material to authorize edits to the list. Instead, it ensures consistency of requested changes over a specified time span (the cool-down period).

9.4 Manifest Freshness

The proposed system currently lacks guarantees regarding the freshness of application manifest metadata. This means administrators could revert to any number of previously signed manifests that are still valid under an authorized policy, potentially allowing attackers who compromise the server to transparently roll back an application to a known vulnerable version. While browsers like Tor or Firefox in incognito mode do not persist state, regular browsing contexts could cache the latest seen manifest version for each signed website and raise a blocking error if a downgrade is detected. Addressing this issue within the current architecture is particularly challenging, as it is intentionally designed to avoid altering the list material during application updates. This minimizes changes and ensures that administrative updates remain seamless. However, as discussed in Section 9.1, if the trust material were instead served directly by the application server, and the list only functioned as a boolean indicator of enrollment, this would open up more flexibility for enforcing freshness guarantees. In this alternative setup, server-side updates could occur more frequently than list updates, enabling tighter control over version rollbacks.

9.5 Manifest or Bundle

Other approaches, particularly those involving signed software packages, typically rely on signed bundles, where the entire package is signed as a whole (see e.g., Section 3.6). This model suits traditional software well, where there is a clear installation phase followed by persistence. However, for web applications that lack a conventional install-and-persist model, this would require end users to always download the full package before any functionality can be loaded, and to verify the entire bundle upfront. While this is acceptable in installation scenarios, since it is a one-time operation, it introduces significant performance issues in the web context. Without persistence, users would be forced to re-download and re-verify the entire bundle on every load. WEBCAT addresses this

problem by supporting dynamic loading of libraries and components. Each component is verified only when it is actually loaded, allowing users to download and verify only the minimal set of assets needed at runtime.

9.6 Infrastructure

The WEBCAT infrastructure represents a critical point of sensitivity and a potential single point of failure from multiple perspectives; this is particularly true of the Enrollment Server and the Build Server. Introducing a new centralized authority in a competitive landscape with diverse and often conflicting interests presents some challenges; Chrome's HSTS preload list, which is only used by Chromium-based browsers, serves as an example.

Both the Enrollment Server and the Build Server are amenable to redesign that would address these concerns, but the architecture requirements would be more complex, and were omitted for the sake of project scope. Neither component imposes technical requirements that mandate a centralized service or administration by a single organization; both components could be redesigned as distributed, consensus-based systems.

9.7 User Interface

While the monitoring of policy changes and enrolled domains is largely delegated to third-party observers, such as advanced users, developers watching known instances, or independent monitors, users still need enough information to make immediate, independent trust decisions. The WEBCAT browser extension fails closed, blocking execution if the web server's signing policy headers don't match the trust list, if the manifest doesn't satisfy the policy, or if the CSP or application components fail integrity checks. However, a malicious administrator could fork a popular web app, insert a backdoor, sign their version of the application with their own valid identities, and enroll their own domain in WEBCAT under a policy they can fully satisfy. If that domain isn't being monitored, users may have no way of knowing they're not using the official version of the application. Even if the domain is being watched, there is an element of due diligence required of users, and warnings or advisories about malicious instances may not always reach users.

Currently, WEBCAT displays the full OIDC identities of verified signatures when the extension icon is clicked. A simpler approach might visually emphasize the domain component, for example by indicating which domain the verified signatures are from (e.g., "verified 3 signatures from cryptpad.org") to help users quickly recognize whether the signature is tied to the original developers.

10 CONCLUSION

The WEBCAT architecture has demonstrated the real-world feasibility of decoupling trust from the server to the application, or individual pages, it serves, in an automated and fail-closed manner. Unlike other approaches, it does not rely on TLS, and accounts for HTTP delivered through different transport modes. The development of WEBCAT defines the requirements for such a system and proposes the first iteration of a possible general-purpose solution. Further research is needed to address open questions, particularly around application freshness and list scalability. Any such mechanism would ultimately require standardization through the W3C

and broader agreement among browser vendors for native support. As a secondary contribution, WEBCAT demonstrates the broader capabilities of the Web Cryptography API by building and releasing native browser components for supply chain integrity and cryptographic logs. Lastly, the efforts in porting existing applications have highlighted the complexity of modern web features and how some of them are used for security purposes in ways not typically considered when designing integrity mechanisms, such as iframe sandboxing and fine-grained CSP applied to workers or specific scripts.

REFERENCES

- [1] Alessandro Acquisti, Idris Adjerid, Rebecca Balebako, Laura Brandimarte, Lorrie Faith Cranor, Saranga Komanduri, Pedro Giovanni Leon, Norman Sadeh, Florian Schaub, Manya Sleeper, Yang Wang, and Shomir Wilson. 2017. Nudges for Privacy and Security: Understanding and Assisting Users' Choices Online. *ACM Comput. Surv.* 50, 3, Article 44 (Aug. 2017), 41 pages. <https://doi.org/10.1145/3054926>
- [2] Carlisle Adams, Patrick Cain, Denis Pinkas, and Robert Zuccherato. 2001. Internet X.509 Public Key Infrastructure Time-Stamp Protocol (TSP). RFC 3161. <https://doi.org/10.17487/RFC3161>
- [3] Bushra A. Alahmadi, Louise Axon, and Ivan Martinovic. 2022. 99% False Positives: A Qualitative Study of SOC Analysts' Perspectives on Security Alarms. In *31st USENIX Security Symposium (USENIX Security 22)*. USENIX Association, Boston, MA, 2783–2800. <https://www.usenix.org/conference/usenixsecurity22/presentation/alahmadi>
- [4] Richard Barnes, J. Hoffman-Andrews, D. McCarney, and J. Kasten. 2019. Automatic Certificate Management Environment (ACME). RFC 8555. <https://doi.org/10.17487/RFC8555>
- [5] Daniel J. Bernstein, Niels Duif, Tanja Lange, Peter Schwabe, and Bo-Yin Yang. 2011. High-speed high-security signatures. *Cryptology ePrint Archive*, Paper 2011/368. <https://eprint.iacr.org/2011/368>
- [6] Henry Birge-Lee, Yixin Sun, Anne Edmundson, Jennifer Rexford, and Prateek Mittal. 2018. Bamboozling Certificate Authorities with BGP. In *27th USENIX Security Symposium (USENIX Security 18)*. USENIX Association, Baltimore, MD, 833–849. <https://www.usenix.org/conference/usenixsecurity18/presentation/birge-lee>
- [7] Jianjun Chen, Jian Jiang, Haixin Duan, Tao Wan, Shuo Chen, Vern Paxson, and Min Yang. 2018. We Still Don't Have Secure Cross-Domain Requests: an Empirical Study of CORS. In *27th USENIX Security Symposium (USENIX Security 18)*. USENIX Association, Baltimore, MD, 1079–1093. <https://www.usenix.org/conference/usenixsecurity18/presentation/chen-jianjun>
- [8] Mauro Cherubini, Alexandre Meylan, Bertil Chapuis, Mathias Humbert, Igor Bilogrevic, and Kévin Huguenin. 2018. Towards Usable Checksums: Automating the Integrity Verification of Web Downloads for the Masses. In *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security (Toronto, Canada) (CCS '18)*. Association for Computing Machinery, New York, NY, USA, 1256–1271. <https://doi.org/10.1145/3243734.3243746>
- [9] Grace H. Cimaszewski, Henry Birge-Lee, Liang Wang, Jennifer Rexford, and Prateek Mittal. 2023. How Effective is Multiple-Vantage-Point Domain Control Validation?. In *32nd USENIX Security Symposium (USENIX Security 23)*. USENIX Association, Anaheim, CA, 5701–5718. <https://www.usenix.org/conference/usenixsecurity23/presentation/cimaszewski>
- [10] Ivan Dolnák and Ján Litvik. 2017. Introduction to HTTP security headers and implementation of HTTP strict transport security (HSTS) header for HTTPS enforcing. In *2017 15th International Conference on Emerging eLearning Technologies and Applications (ICETA)*. 1–4. <https://doi.org/10.1109/ICETA.2017.8102478>
- [11] Adam Doupe, Weidong Cui, Mariusz H. Jakubowski, Marcus Peinado, Christopher Kruegel, and Giovanni Vigna. 2013. deDacota: toward preventing server-side XSS via automatic code and data separation. In *Proceedings of the 2013 ACM SIGSAC Conference on Computer & Communications Security (Berlin, Germany) (CCS '13)*. Association for Computing Machinery, New York, NY, USA, 1205–1216. <https://doi.org/10.1145/2508859.2516708>
- [12] Harry Halpin. 2014. The W3C web cryptography API: motivation and overview. In *Proceedings of the 23rd International Conference on World Wide Web (Seoul, Korea) (WWW '14 Companion)*. Association for Computing Machinery, New York, NY, USA, 959–964. <https://doi.org/10.1145/2567948.2579224>
- [13] Richard Hansen and Vicente Silveira. 2022. Code Verify: An Open Source Browser Extension for Verifying Code Authenticity on the Web. <https://engineering.fb.com/2022/03/10/security/code-verify/>. Accessed: 2025-05-04.
- [14] Daniel Huygens. 2023. Source Code Transparency Explainer. <https://github.com/twiss/source-code-transparency/blob/main/explainer.md>. Accessed: 2025-05-04.
- [15] Rob Jansen, Tavish Vaidya, and Micah Sherr. 2019. Point Break: A Study of Bandwidth Denial-of-Service Attacks against Tor. In *28th USENIX Security Symposium (USENIX Security 19)*. USENIX Association, Santa Clara, CA, 1823–1840. <https://www.usenix.org/conference/usenixsecurity19/presentation/jansen>
- [16] Hariharan Jayakrishnan and Ritwik Murali. 2019. A Simple and Robust End-to-End Encryption Architecture for Anonymous and Secure Whistleblowing. In *2019 Twelfth International Conference on Contemporary Computing (IC3)*. 1–6. <https://doi.org/10.1109/IC3.2019.8844917>
- [17] Nikolaos Karapanos, Alexandros Filios, Raluca Ada Popa, and Srdjan Capkun. 2016. Verena: End-to-End Integrity Protection for Web Applications. In *2016 IEEE Symposium on Security and Privacy (SP)*. 895–913. <https://doi.org/10.1109/SP.2016.58>
- [18] James Larisch, David Choffnes, Dave Levin, Bruce M. Maggs, Alan Mislove, and Christo Wilson. 2017. CRLite: A Scalable System for Pushing All TLS Revocations to All Browsers. In *2017 IEEE Symposium on Security and Privacy (SP)*. 539–556. <https://doi.org/10.1109/SP.2017.17>
- [19] Dominik Meißner, Frank Kargl, and Benjamin Erb. 2021. WAIT: protecting the integrity of web applications with binary-equivalent transparency. In *Proceedings of the 36th Annual ACM Symposium on Applied Computing (SAC '21)*. ACM, 1950–1953. <https://doi.org/10.1145/3412841.3442143>
- [20] Marcela S. Melara, Aaron Blankstein, Joseph Bonneau, Edward W. Felten, and Michael J. Freedman. 2015. CONIKS: bringing key transparency to end users. In *Proceedings of the 24th USENIX Conference on Security Symposium (Washington, D.C.) (SEC'15)*. USENIX Association, USA, 383–398.
- [21] Stefania Milan. 2015. Hacktivism as a Radical Media Practice. In *The Routledge Companion to Alternative and Community Media*, Chris Atton (Ed.). Routledge, 550–560.
- [22] Niels Möller and Rasmus Dahlberg. 2023. Sigsum: A Minimal Design for Public Transparency Logs. <https://www.sigsum.org/> Supplementary notes accompanying the CATS 2023 lightning talk.
- [23] Zachary Newman, John Speed Meyers, and Santiago Torres-Arias. 2022. Sigstore: Software Signing for Everybody. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security (Los Angeles, CA, USA) (CCS '22)*. Association for Computing Machinery, New York, NY, USA, 2353–2367. <https://doi.org/10.1145/3548606.3560596>
- [24] Raluca Ada Popa, Emily Stark, Steven Valdez, Jonas Helfer, Nickolai Zeldovich, and Hari Balakrishnan. 2014. Building Web Applications on Top of Encrypted Data Using Mylar. In *11th USENIX Symposium on Networked Systems Design and Implementation (NSDI 14)*. USENIX Association, Seattle, WA, 157–172. <https://www.usenix.org/conference/nsdi14/technical-sessions/presentation/popa>
- [25] Justin Samuel, Nick Mathewson, Justin Capps, and Roger Dingledine. 2010. Survivable key compromise in software update systems. In *Proceedings of the 17th ACM Conference on Computer and Communications Security (Chicago, Illinois, USA) (CCS '10)*. Association for Computing Machinery, New York, NY, USA, 61–72. <https://doi.org/10.1145/1866307.1866315>
- [26] Quirin Scheitle, Oliver Gasser, Theodor Nolte, Johanna Amann, Lexi Brent, Georg Carle, Ralph Holz, Thomas C. Schmidt, and Matthias Wählisch. 2018. The Rise of Certificate Transparency and Its Implications on the Internet Ecosystem. In *Proceedings of the Internet Measurement Conference 2018 (Boston, MA, USA) (IMC '18)*. Association for Computing Machinery, New York, NY, USA, 343–349. <https://doi.org/10.1145/3278532.3278562>
- [27] Rajiv C. Shah and Christian Sandvig and. 2008. SOFTWARE DEFAULTS AS DE FACTO REGULATION The case of the wireless internet. *Information, Communication & Society* 11, 1 (2008), 25–46. <https://doi.org/10.1080/13691180701858836> arXiv:<https://doi.org/10.1080/13691180701858836>
- [28] Ewa Syta, Iulia Tamas, Dylan Visher, David Isaac Wolinsky, Philipp Jovanovic, Linus Gasser, Nicolas Gailly, Ismail Khoffi, and Bryan Ford. 2016. Keeping Authorities "Honest or Bust" with Decentralized Witness Cosigning. In *2016 IEEE Symposium on Security and Privacy (SP)*. 526–545. <https://doi.org/10.1109/SP.2016.38>
- [29] Web Incubator Community Group (WICG). 2023. Isolated Web Apps Explainer. <https://github.com/WICG/isolated-web-apps/blob/main/README.md>. Accessed: 2025-05-04.
- [30] Lukas Weichselbaum, Michele Spagnuolo, Sebastian Lekies, and Artur Janc. 2016. CSP Is Dead, Long Live CSP! On the Insecurity of Whitelists and the Future of Content Security Policy. In *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security (Vienna, Austria) (CCS '16)*. Association for Computing Machinery, New York, NY, USA, 1376–1387. <https://doi.org/10.1145/2976749.2978363>
- [31] Jie Xu, Cong Wang, and Xiaohua Jia. 2023. A Survey of Blockchain Consensus Protocols. *ACM Comput. Surv.* 55, 13s, Article 278 (July 2023), 35 pages. <https://doi.org/10.1145/3579845>
- [32] Bryant Zadegan and Ryan Lester. 2016. Abusing Bleeding Edge Web Standards for AppSec Glory. In *DEF CON 24 Hacking Conference*. <https://media.defcon.org/DEF%20CON%2024/DEF%20CON%2024%20presentations/DEF%20CON%2024%20-%20Bryant-Zadegan-Ryan-Lester-Abusing-Bleeding-Edge-Web-Standards-For-Appsec-Glory.pdf> Accessed: 2025-04-02.

A ARTIFACT APPENDIX

A.1 Abstract

The WEBCAT artifacts adhere to the logical structure and separation of components of the project. They include the WEBCAT browser extension and its dependencies, a prototype of the enrollment server API and processor, a list update builder and signing worker, and a signing script for web application developers, together constituting all the core components needed to run the system. Additionally, two extra scripts are provided: one for benchmarking the browser extension, and another to analyze the resulting data and generate the charts.

A.2 Description & Requirements

The system setup has a client-server architecture, and relies on third party hosted infrastructure, which backend services are fully open source, but which hosting is out-of-scope for the evaluation. The browser extension and the benchmarking script can be run with user privileges, given the needed dependencies are installed, and the same applies to the server components.

A.2.1 Security, privacy, and ethical concerns. No specific security, or ethical concerns exists for running the evaluation. When using the web application signing script, information such as the OIDC identity used for signing will be logged in the append-only Rekor transparency log, part of the Sigstore *Public Good* instance. Information about signing identities and artifacts cannot be removed and is automatically published as part of the signing operation. The browser extension has not been audited and might increase attack surface or contain vulnerabilities; it is thus discouraged to use it outside the scope of this document.

A.2.2 How to access. The Git repository is available at <https://github.com/lsd-cat/webcat>, commit 2458e9e. It is a *monorepo* that contains all the required components, including the database with the collected benchmark results.

A.2.3 Hardware dependencies. No specific hardware is required; however, all tests have been conducted exclusively on the *amd64* architecture. Prolonged benchmarking requires either periodic cleanup or ample disk space, as each iteration consumes approximately 30MB of cache storage. For instance, 3000 iterations may require up to 100GB of available disk space. Building the project requires minimal resources: 2GB of RAM and a single CPU core are sufficient. For realistic benchmarking, a desktop machine is recommended; an Intel i7-7700 with 64GB of RAM was used in testing.

A.2.4 Software dependencies. Everything has been compiled and tested on Debian 12.10. Previous versions and other Linux distributions could work, but they have not been tested. The benchmark has been executed on Ubuntu 24.04.2.

A.2.5 Benchmarks. The benchmark has been run on the *Element* web application, version 1.11.94. A backup of that version, with the addition of the signed *webcat.json* manifest file needed for testing, is available at <https://github.com/lsd-cat/webcat-element> and is included as a git submodule.

A.3 Set-up

A.3.1 Installation. Run as root the following commands to install the required dependencies. The latest tested Firefox version is 138.0.1. The minimum required Firefox version is 134.

```
apt update -y
apt install -y git npm python3 python3-venv \
    firefox zip build-essential
```

Manually install Go 1.24.2, as Debian repositories contain an older, non-compatible version. The following commands can be run as a unprivileged user.

```
wget https://go.dev/dl/go1.24.2.linux-amd64.tar.gz
mkdir -p ~/.local/go
tar -C ~/.local -xzf go1.24.2.linux-amd64.tar.gz
echo 'export PATH=$PATH:$HOME/.local/go/bin' >> ~/.profile
source ~/.profile
```

As the same unprivileged user, clone and build the various components.

```
git clone --recurse-submodules https://github.com/lsd-cat/webcat
cd webcat
make
```

A.3.2 Basic Test. Check that the builds were successful, *webcat/dist* should contain:

- *webcat-extension.zip*
- *webcat-extension-test.zip*
- *webcat-build-server.zip*
- *webcat-list-server.zip*

Install the browser extension in Firefox:

- (1) Open a new tab
- (2) Navigate to `about:debugging`
- (3) In the left menu, click *This Firefox*
- (4) Click on *Load Temporary Add-on...*
- (5) Select *webcat-extension.zip*
- (6) Verify that the extension loads and click *Inspect*
- (7) Check the Developer Console for errors

A.4 Evaluation workflow

A.4.1 Major Claims.

- (C1): WEBCAT is able to verify and enforce the integrity of web applications according to the model described.
- (C2): WEBCAT has minimal (<5%) impact of non-enrolled domains, and a low impact on enrolled ones (<25%). *Note: benchmarking has a lot of variability, especially if the browser or OS are running other tasks and are long-lived.*

A.4.2 Experiments. We present two experiments, one to show the integrity enforcement, and one to measure the performance.

- (E1): Functional tests [5 human-minutes + 10 compute-minutes + 5GB disk]: verify that the WEBCAT extension can detect and block anomalies in signing policy, signing threshold, manifest corruption, headers stripping, html and JS files corruption.

How to: Run the set-up as described before. Ensure the test machine has a graphical environment and is able to execute *firefox*.

Execution: Run the following command.

```
make test
```

Results: At the end, the following result should be printed.
collected 11 items

```
test_webcat[basic_test] PASSED          [ 9%]
test_webcat[stripping_test] PASSED      [ 18%]
test_webcat[wrong_threshold_test] PASSED [ 27%]
test_webcat[wrong_signers_test] PASSED  [ 36%]
test_webcat[wrong_csp_test] PASSED      [ 45%]
test_webcat[missing_threshold_test] PASSED [ 54%]
test_webcat[missing_signers_test] PASSED [ 63%]
test_webcat[missing_csp_test] PASSED    [ 72%]
test_webcat[corrupted_index_test] PASSED [ 81%]
test_webcat[corrupted_manifest_test] PASSED [ 90%]
test_webcat[corrupted_js_test] PASSED   [100%]
```

```
===== 11 passed in 334.90s (0:05:34) =====
```

(E2): [Extension benchmark] [5 human-minutes + 5 compute-minute + 1G disk]: run 5 iterations for every benchmark case: cold and warm startup, with enrolled host, non-enrolled host and without the extension.

How to: Run the set-up as described before. Ensure the test machine has a graphical environment and is able to execute *firefox*.

Execution: Run the following command.

```
make benchmark
```

Results: The terminal should print the following results, and output an sqlite3 database in test/results.db.

Test URL	Enrolled	Addon	#	Time
http://127.0.0.1:59560/	true	true	5/5	137.4s
http://localhost:60197/	false	true	5/5	136.1s
http://127.0.0.1:60828/	false	false	5/5	134.5s

A.5 Version

Based on the LaTeX template for Artifact Evaluation V20231005. Submission, reviewing and badging methodology followed for the evaluation of this artifact can be found at <https://secartifacts.github.io/usenixsec2024/>.